4. IMPLEMENTATION

ProjectForge AI is implemented as a software-development platform that converts a user's requirements into structured analysis, a build specification, generated source files, and verification results. The platform uses a React and TypeScript frontend and a FastAPI backend. A locally hosted language model can be accessed through LM Studio.

The development workflow is divided among six specialist roles: Product, AI/ML, Engineering, UX/UI, Security, and Quality Assurance. These roles use different instructions and output contracts. They do not require six separately downloaded language models. In the local configuration, specialist tasks are executed sequentially to limit memory consumption.

ProjectForge combines model-produced specifications with reviewed reusable generation modules. Therefore, generated applications contain source code produced from validated specifications and reusable implementation components. The system does not assume that unrestricted model output is correct or safe.

ProjectForge stores its project snapshots, history, and execution events in SQLite. Generated applications maintain their own databases. The simple CRUD examples use SQLite, while the commerce demonstrations use PostgreSQL.

The implementation described in this chapter includes the ProjectForge platform, generated CRUD examples, and the prepared commerce demonstrations. Features requiring further external verification are identified separately.

4.1 Screen Layouts with Validations

4.1.1 Project Creation Screen

The project creation screen allows the user to describe the application to be developed. The description identifies the application's purpose, entities, required operations, technology preferences, and acceptance criteria.

The submitted description becomes the starting point for requirement understanding. Where information is incomplete, the system can request clarification before proceeding to specialist analysis.

Validation and behaviour:

- The backend validates the project request.
- Incomplete requirements are represented as clarification items.
- Specialist analysis requires an acceptable requirements state.
- A successful request submission does not indicate that application generation has completed.

[Insert the New Project form containing a Task Manager or another supported application description.]

Figure 4.1: Project creation and requirement submission.

4.1.2 Requirement Understanding and Clarification Screen

The requirement screen presents the system's interpretation of the user's request. It enables the user to review extracted information and answer outstanding clarification questions.

The project context preserves requirements and their supporting information. When model understanding fails, labelled heuristic suggestions may be provided. These suggestions must not be presented as validated model analysis.

Validation and behaviour:

- Outstanding clarification questions must be resolved before analysis proceeds.
- User answers update the project context.
- Derived suggestions remain distinguishable from explicit user requirements.
- Model errors are recorded rather than silently treated as successful understanding.

[Insert extracted requirements and a clarification question or completed clarification response.]

Figure 4.2: Requirement review and clarification.

4.1.3 Virtual Agent Office

The virtual office provides a visual representation of the development team. Each specialist is associated with a role, workstation, status, and task information.

The office helps users understand the current stage of execution. The activity stream displays actual application events, including task execution, validation failures, completion messages, and stopped runs.

Validation and behaviour:

- Agent status is associated with backend execution information.
- A failed operation must remain visible.
- Completion of specialist analysis does not automatically mean that generated code has passed testing.
- Visual animation is a presentation feature and is not proof of background work.

[Insert the office with agent statuses and the activity stream visible.]

Figure 4.3: Virtual agent office and execution activity.

4.1.4 Agent Details and Blueprint Screen

The agent details view allows the user to inspect a specialist's assignment and available output. The blueprint view presents the combined planning information produced from specialist results.

A structured build specification describes supported implementation details and acceptance requirements. The build stage checks that the specification is valid and applicable to the current project version.

Validation and behaviour:

- Specialist responses must satisfy their required output structure.
- Invalid responses are subject to bounded retry and validation.
- Missing approval or an outdated specification can prevent execution.
- A planning blueprint is not equivalent to verified source code.

[Insert one agent's actual output and the corresponding project blueprint.]

Figure 4.4: Specialist output and project blueprint.

4.1.5 Build and Verification Results

The build stage generates source files and executes the supported verification checks. Results distinguish backend tests, frontend compilation, acceptance checks, and browser journeys.

An application may pass backend tests but fail a browser journey. Such a result must not be described as completely verified.

Validation and behaviour:

- Results are obtained from executed checks.
- Failed checks remain visible in the build report.
- Unsupported or unresolved failures may require human review.
- Verification applies to the tested source version and declared cases.

[Insert a build report showing individual verification checks and the final status.]

Figure 4.5: Build execution and verification results.

4.1.6 Generated Storefront and Product Details

The commerce demonstration contains a product catalogue with categories, product variants, prices, stock information, search, and filtering. Product details allow the customer to inspect a selected product and its available options.

Product information is obtained from the application backend. The current demonstration uses reviewed local image assets.

Validation and behaviour:

- Inactive catalogue items are excluded from public results.
- Search and filter parameters are validated.
- Price and stock values are constrained by backend and database rules.
- Arbitrary file uploads and remote image fetching are outside the current demonstration scope.

[Insert the desktop product catalogue.]

Figure 4.6: Generated commerce product catalogue.

[Insert a product detail page with variant selection.]

Figure 4.7: Product details and variant selection.

4.1.7 Customer Account and Address Screens

The account interface supports registration, sign-in, profile information, password recovery, and customer addresses.

Authentication and ownership checks are enforced by the backend. A customer must not gain access to another customer's records by changing an identifier in a request.

Validation and behaviour:

- Registration fields are validated.
- Passwords are stored as salted hashes.
- Protected operations require an authenticated session.
- Address operations are restricted to the owning customer.

- Password-reset tokens expire and cannot be reused after successful consumption.

[Insert the account screen and an address form, including one visible validation error.]

Figure 4.8: Customer account, address management, and validation.

4.1.8 Cart and Checkout Screens

The cart displays selected variants and quantities. During checkout, the backend calculates prices and totals and creates an order.

Cart contents do not reserve stock. Stock is reserved during checkout through a database transaction. The system uses request identifiers to prevent duplicate order creation during retries.

Validation and behaviour:

- Quantities must satisfy supported limits.
- Available stock is checked during checkout.
- Client-provided prices are not authoritative.
- Order totals are calculated on the server.
- Repeated requests with the same valid idempotency key must not create duplicate orders.

[Insert a cart containing products and the resulting awaiting-payment order.]

Figure 4.9: Cart and server-validated checkout.

4.1.9 Order History and Offline Payment Simulation

Customers can view their orders and associated states. The offline payment simulator allows controlled demonstrations of pending, successful, failed, and cancelled payment outcomes.

These outcomes represent simulation records. No actual money is collected.

Validation and behaviour:

- Customers can access only their own orders.
- Payment amounts and currency are derived from the stored order.
- Replayed events must not repeat financial or stock effects.
- Contradictory terminal outcomes are rejected.
- Simulated refunds require administrator authorization.

[Insert order history and a pending payment simulation.]

Figure 4.10: Order history and pending payment simulation.

[Insert a clearly labelled successful payment simulation.]

Figure 4.11: Successful offline payment simulation.

[Insert an administrator's simulated refund.]

Figure 4.12: Authorized simulated refund.

4.1.10 Administration and Responsive Layout

The administration interface provides authorized catalogue, stock, and order operations. Customer accounts cannot obtain administrator privileges through frontend navigation alone.

The commerce frontend also provides layouts suitable for smaller screens. Browser checks cover selected keyboard interactions and desktop/mobile layout conditions.

Validation and behaviour:

- Administrator permissions are enforced by the backend.
- Invalid stock and price values are rejected.
- Conflicting catalogue updates require review or reload.
- Responsive layout checks do not establish full accessibility compliance.

[Insert the administrator catalogue or stock screen.]

Figure 4.13: Authorized catalogue and stock administration.

[Insert a mobile storefront screen.]

Figure 4.14: Responsive mobile storefront.

4.1.11 Validation Summary

Table 4.1: Principal application validations.

Area	Validation or control	Expected behaviour
Requirements	Outstanding clarification items	Analysis is prevented until the required information is resolved.
Agent output	Required fields and decision structure	Invalid output is retried within configured limits or reported.
Build specification	Approval and project-version checks	Missing approval or incompatible specifications prevent execution.
Registration	Field and password validation	Invalid registration is rejected.
Authentication	Session validation	Protected operations require sign-in.
Addresses and orders	Object ownership	Another customer's records cannot be accessed.
Administration	Role validation	Customer requests to administrator operations are rejected.
Mutating requests	Origin and CSRF validation	Invalid protected requests are rejected.
Product data	Price, stock, SKU, and relationship constraints	Invalid data cannot be accepted as a valid catalogue update.
Checkout	Server calculation	Browser price changes do not determine the order total.
Inventory	Transactional reservation	Checkout must not reserve unavailable quantities.

Area	Validation or control	Expected behaviour
Request retries	Idempotency checks	A retry does not create an additional order or repeated effect.
Payments	Signature and provider-state checks	Untrusted payment claims do not establish payment success.
Password recovery	Expiry and single-use tokens	Expired or consumed tokens are rejected.
Verification	Separate result categories	Passing one check does not hide a failure in another.

4.2 Sample Coding with Security Tasks

The following excerpts illustrate implemented persistence and security controls. They belong to larger modules and are not standalone applications.

4.2.1 Parameterized Database Operations

ProjectForge stores project snapshots and history using parameterized SQLite operations.

```
previous = db.execute(
    "SELECT payload FROM snapshots WHERE project_id=?",
    (project_id,)
).fetchone()

if previous and previous[0] == payload:
    return

db.execute(
    "INSERT OR REPLACE INTO snapshots VALUES (?, ?)",
    (project_id, payload)
)

db.execute(
    "INSERT INTO history(project_id, payload) VALUES (?, ?)",
    (project_id, payload)
)
```

Explanation: The project identifier and serialized payload are passed as parameters. They are not concatenated into SQL statements. The latest project state is stored in the snapshot table, while changed states are also recorded in history.

Security task: Separate data from SQL syntax to reduce SQL-injection risk in these operations.

Source: backend/app/api/persistence.py

4.2.2 Password Hashing

The generated identity module uses the scrypt password-hashing function with a random salt.

```
  def hash_password(password):
      salt = secrets.token_bytes(16)

      with HASH_GATE:
          value = hashlib.scrypt(
              password.encode(),
              salt=salt,
              n=131072,
              r=8,
              p=1,
              dklen=32,
              maxmem=256 * 1024 * 1024
          )

      return (
          "scrypt$131072$8$1$"
          + salt.hex()
          + "$"
          + value.hex()
      )
```

Explanation: Each password receives a random salt. The database stores the encoded hash parameters, salt, and derived value rather than the original password. Password verification derives a new value and compares it using a constant-time comparison.

Security task: Reduce the exposure of original passwords if stored authentication data is compromised.

Source: docs/ecommerce-evidence/payments-app/backend/app/identity_runtime.py

4.2.3 Administrator Authorization

The backend checks the authenticated user's role before allowing administrator operations.

```
  def require_admin(request):
      _, user = session(request, True)

      if user["role"] != "admin":
          raise HTTPException(
              403,
              "Administrator role required"
          )

      return user
```

Explanation: The function requires an authenticated session and then checks the server-side role. Hiding an administrator button in the frontend is insufficient; authorization must be checked for the corresponding backend operation.

Security task: Prevent ordinary customers from performing privileged administration actions.

Source: docs/ecommerce-evidence/payments-app/backend/app/identity_runtime.py

4.2.4 CSRF and Origin Validation

Protected mutation requests are checked against the configured origin and the session's CSRF token.

```
    if request.headers.get("origin") != origin:
        raise HTTPException(
            403,
            "Request origin is not allowed"
        )

    if request.headers.get("sec-fetch-site") == "cross-site":
        raise HTTPException(
            403,
            "Cross-site request rejected"
        )

    row, user = session(request, authenticated)
    supplied = request.headers.get("x-csrf-token", "")

    if len(supplied) != 43 or not hmac.compare_digest(
        supplied,
        row["csrf"]
    ):
        raise HTTPException(
            403,
            "CSRF token is missing or invalid"
        )
```

Explanation: The backend checks the request origin and compares the submitted token with the session token. Invalid requests receive an HTTP 403 response.

Security task: Protect authenticated state-changing operations against cross-site request forgery.

Source: docs/ecommerce-evidence/payments-app/backend/app/identity_runtime.py

4.2.5 Payment Signature Verification

The Razorpay test adapter contains an HMAC signature-verification function.

```
def verify_signature(raw, signature, secret):
    if (
        not isinstance(signature, str)
        or not re.fullmatch(r"[0-9a-f]{64}", signature)
    ):
        return False

    expected = hmac.new(
        secret.encode(),
        raw,
        hashlib.sha256
    ).hexdigest()

    return hmac.compare_digest(expected, signature)
```

Explanation: The function calculates an expected signature from the message and server-held secret. It compares the expected and received signatures without treating a client-side success flag as payment evidence.

Signature verification is only one part of payment validation. The complete flow must also match the expected order, amount, currency, account, and payment state.

Security task: Reject forged or modified payment messages and prevent unverified claims of payment success.

Source: docs/ecommerce-evidence/razorpay-app/backend/app/razorpay_client.py

4.2.6 Generated-Code Execution Controls

Generated applications are verified in a restricted Docker environment. The sandbox applies controls such as a non-root user, limited resources, restricted filesystem access, dropped capabilities, and disabled external networking for offline checks.

Security task: Limit the access and resources available to generated code during verification.

Sandboxing reduces exposure but does not establish that generated applications are free from vulnerabilities. Application security testing and deployment controls remain separate requirements.

4.2.7 Recorded Verification Results

The following results describe the recorded security-repair checks dated 24 October 2026. They are not a claim that every possible application behaviour has been tested.

Table 4.2: Recorded verification evidence.

Verification area	Recorded result	Interpretation
Identity regression suite	11 tests passed	Includes password-reset isolation and persistence checks.

Verification area	Recorded result	Interpretation
Razorpay synthetic control suite	14 tests passed	Tests used simulated provider responses.
Razorpay demonstration browser journey	1 test passed	Actual external Razorpay integration remains unverified.
Offline payment control suite	10 tests passed	Covers selected simulation, replay, ownership, and refund controls.
Offline payment demonstration browser journey	1 test failed	Registration completion was not observed within the assertion timeout; investigation remains pending.
Both commerce frontend production builds	Passed	Confirms compilation for the tested versions.
Both commerce frontend dependency audits	Zero reported vulnerabilities in those scans	Does not establish complete application security.
Actual Razorpay account integration	Unverified	Requires external provider testing.
Production load and complete security assessment	Not completed	Must be evaluated separately.

The latest failed browser result is retained as an unresolved issue. It must not be removed from the report or reclassified as passed without supporting evidence.

4.3 Steps of the Execution Project and Navigation

4.3.1 Execution Prerequisites

The local environment requires:

1. The ProjectForge repository and installed project dependencies.
2. Python and the configured virtual environment.
3. Node.js and frontend dependencies.
4. LM Studio with the selected model downloaded.
5. Docker Desktop with its Linux engine running.
6. Required verification images available locally.
7. PostgreSQL for the commerce demonstration.

Offline operation requires the necessary model files, packages, browser components, and container images to be downloaded beforehand. External Razorpay testing requires internet connectivity.

4.3.2 Starting the Local Model

Open a PowerShell terminal and load the configured model:

```
lms load nvidia/nemotron-3-nano-4b --parallel 1 -y
lms server start --port 1234 --bind 127.0.0.1
lms ps
```

The output should confirm that the model is loaded and the local server is available.

The specialist roles share the configured model service. Local inference capacity depends on available RAM, GPU memory, context length, and output limits.

4.3.3 Starting the ProjectForge Backend

Open a separate terminal:

```
cd D:\capstone_project\backend

..\.venv\Scripts\python.exe -m uvicorn app.main:app `
  --host 127.0.0.1 --port 8000
```

Keep this terminal running. It displays backend requests and execution messages.

4.3.4 Starting the ProjectForge Frontend

Open another terminal:

```
cd D:\capstone_project\frontend

npm run dev -- --host 127.0.0.1 --port 3000 --strictPort
```

Open the application at http://127.0.0.1:3000/.

[Insert the running ProjectForge frontend with its browser address visible.]

Figure 4.15: ProjectForge running in the local environment.

4.3.5 Project Navigation and Generation

The user follows this sequence:

1. Select New Project.
2. Enter the application requirements.
3. Review the extracted requirements.
4. Answer pending clarification questions.
5. Start specialist analysis.
6. Inspect the six agents' available outputs.
7. Review the blueprint and build specification.
8. Provide approval when required by the selected workflow.
9. Start the build.
10. Inspect file-generation and verification results.
11. Open the generated source code.

12. Run and manually review the generated application.

The system uses checkpoints and persisted state to support recovery. Resuming an interrupted operation is explicit and depends on the validity of the saved project state.

4.3.6 Locating Generated Source Code

Generated project versions are stored under:

```
D:\capstone_project\backend\generated_projects\<project_id>\v<version>
```

For example, the previously generated NoteKeeper version is located at:

```
D:\capstone_project\backend\generated_projects\PROJ-3D627C8F607B\v2
```

A generated workspace contains files such as:

```
backend/
    app/
    tests/
    requirements.txt
frontend/
    src/
    package.json
    index.html
README.md
```

The exact files depend on the selected generation strategy and supported application scope.

[Insert the generated workspace in VS Code showing the frontend and backend folders and an open source file.]

Figure 4.16: Generated application source code in VS Code.

4.3.7 Running the Prepared Commerce Demonstration

The offline payment demonstration is stored at:

```
D:\capstone_project\docs\ecommerce-evidence\payments-app
```

This is a prepared generated demonstration used for implementation and verification evidence. It should not be presented as proof that every new commerce prompt completes the entire local-model pipeline successfully.

For an already configured installation, start PostgreSQL and set DATABASE_URL privately in the backend terminal. Apply the migrations:

```
cd D:\capstone_project\docs\ecommerce-evidence\payments-app\backend

.\.venv\Scripts\python.exe migrate.py
```

Start the backend in the same terminal:

```
.\.venv\Scripts\python.exe -m uvicorn app.main:app `
    --host 127.0.0.1 --port 8001 --no-proxy-headers
```

In another terminal, start the frontend:

```
cd D:\capstone_project\docs\ecommerce-evidence\payments-app\frontend

npm run dev -- --host 127.0.0.1 --port 3001 --strictPort
```

Open http://127.0.0.1:3001/.

Table 4.3: Commerce demonstration navigation.

Route	Purpose
/	Browse the product catalogue.
/account	Register, sign in, and manage account information.
/cart	Review selected products and initiate checkout.
/orders	View owned orders and offline payment simulations.
/admin	Perform authorized catalogue and stock operations.
/admin/orders	Review orders and perform supported simulated refunds.

Administrator access requires an explicitly created administrator account.

4.3.8 Demonstration Procedure

A suitable demonstration sequence is:

1. Browse the catalogue.
2. Search for or filter a product.
3. Open its product details.
4. Register and sign in using a demonstration account.
5. Add an address.
6. Add a product to the cart.
7. Complete checkout.
8. Observe the awaiting-payment order.
9. Start the clearly labelled payment simulation.
10. Demonstrate one successful or failed simulation.
11. Review order history.
12. Sign in separately as an administrator to demonstrate stock management or a simulated refund.

Actual payment collection, shipping, and fulfillment must not be claimed from these simulation screens.

4.3.9 Screenshot Preparation Checklist

The screenshots listed below are available as historical demonstration evidence in D:\capstone_project\docs\ecommerce-evidence. Use them where they match the description, or

capture new screenshots to show the current running version. Figure placeholders in Section 4.1 identify where each image belongs.

Table 4.4: Available commerce screenshots.

Screenshot	Filename
Desktop catalogue	catalogue-desktop-catalogue.png
Mobile product details	catalogue-mobile-detail.png
Customer account	identity-desktop-account.png
Administrator interface	identity-desktop-admin.png
Cart	orders-desktop-cart.png
Mobile order history	orders-mobile-history.png
Pending payment simulation	payments-desktop-pending.png
Successful payment simulation	payments-mobile-success.png
Simulated refund	payments-desktop-refund.png
Mobile catalogue	catalogue-mobile-catalogue.png

Capture additional ProjectForge screenshots for project creation, requirements and clarification, the agent office, specialist output and blueprint, the verification report, the running application, and generated code in VS Code. Capture a validation error and address form if the existing account image does not show them.

Keep passwords, API secrets, session tokens, and personal customer details out of screenshots. Replace each screenshot placeholder with the corresponding image and retain its numbered caption.

5. CONCLUSION

ProjectForge AI demonstrates a structured approach to assisted software development. It connects requirement understanding, specialist analysis, build planning, source generation, and verification within a single workflow.

The project's contribution is the coordination and validation around model use. A model response is treated as an intermediate result that must satisfy a contract, rather than being accepted automatically as correct software.

The implementation supports constrained application generation and prepared commerce modules. The commerce demonstrations include catalogue, identity, cart, order, inventory-reservation, and payment-simulation behaviour. A separate Razorpay test adapter has been implemented and tested with synthetic provider responses.

The recorded results demonstrate useful functionality and selected security controls. However, unresolved browser verification, external payment testing, deployment hardening, and broader security assessment remain. The project is therefore presented as a development and verification prototype rather than a production-certified commerce platform.

5.1 Importance of the Worked-Out Tasks

5.1.1 Structured Requirement Understanding

The requirement workflow reduces ambiguity before implementation. Clarification questions and explicit acceptance criteria help align generated functionality with user expectations.

5.1.2 Specialist Analysis

The six specialist roles separate product, engineering, design, AI suitability, security, and testing concerns. This makes decisions easier to inspect and reduces dependence on a single unrestricted response.

5.1.3 Controlled Local Model Use

LM Studio enables supported inference tasks to run locally after the model has been downloaded. Sequential execution and bounded requests make the workflow more practical on limited hardware.

Offline model access does not remove the need for validation or guarantee that every application can be generated successfully.

5.1.4 Reusable and Inspectable Source Generation

Reviewed generation modules provide consistent implementations for supported patterns. The resulting source files remain available for inspection, editing, and further development.

This improves explainability because users can examine the actual application code rather than receiving only a visual preview.

5.1.5 Persistence and Recovery

Stored snapshots, history, events, and checkpoints help retain project progress and explain failures. Recovery can reuse valid completed work when supported by the current state.

5.1.6 Independent Verification

Separating source generation from verification prevents the system from equating "files created" with "application works." Backend checks, frontend builds, browser journeys, and business-rule tests provide different forms of evidence.

5.1.7 Commerce Business Rules

Server-calculated totals, integer monetary values, transactional inventory reservations, ownership checks, and request idempotency address important commerce correctness requirements.

5.1.8 Security Awareness

The implementation includes password hashing, session controls, CSRF protection, role checks, payment-message validation, and restricted execution.

The security review also identified remaining weaknesses. Recording these limitations supports responsible engineering and provides a clear improvement plan.

5.2 Further Enhancement in Future

Table 5.1: Future enhancement priorities.

Enhancement	Proposed work	Expected benefit
Complete browser reliability	Diagnose the outstanding simulator registration/browser failure and rerun the unchanged acceptance journey.	More dependable demonstrations and verification.
Database least privilege	Separate database bootstrap, migration, and runtime roles.	Reduce the impact of an application compromise.
Durable webhook processing	Persist verified incoming events before bounded background processing.	Improve payment-event reliability during slow provider responses or restarts.
Actual provider verification	Complete Razorpay test-account checkout, webhook, reconciliation, and refund testing.	Establish evidence beyond mocked provider behaviour.
Production browser security	Configure and test CSP, framing restrictions, HTTPS, and deployment headers.	Improve protection in public deployments.

Enhancement	Proposed work	Expected benefit
Dependency and container maintenance	Expand dependency, installer, operating-system, and container scans.	Reduce exposure to known vulnerabilities.
Broader security assessment	Add systematic access-control, XSS, abuse, and ASVS-based checks.	Improve security coverage.
Local-model commerce validation	Verify a fresh supported commerce request through the complete model-to-build workflow.	Demonstrate reproducible generation beyond prepared fixtures.
Additional generation patterns	Extend supported entities, relationships, workflows, and migrations.	Support a wider range of applications.
Operational commerce features	Add reviewed fulfillment, returns, and partial-refund workflows.	Support more complete store operations.
Accessibility and performance	Expand keyboard, assistive-technology, mobile, and load testing.	Improve usability and establish measured capacity.
Deployment and recovery	Add backup restoration tests, monitoring, and deployment procedures.	Improve operational reliability.

Future features should be introduced in independently verifiable phases. A phase should be marked complete only when its relevant acceptance checks pass.

6. REFERENCES

The following references support the technologies, architectural concepts, and verification practices described in this report. Official documentation is used for implementation guidance. The research paper is related background literature and does not imply that ProjectForge directly depends on that framework.

6.1 Books, Research Papers, Experts Persons, Technology, Webs, etc.

Research Paper

1. Wu, Q., et al. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. Available at: https://arxiv.org/abs/2308.08155.

Official Technology Documentation

2. React Team. React Documentation: Quick Start. Available at: https://react.dev/learn.
3. FastAPI. FastAPI Documentation. Available at: https://fastapi.tiangolo.com/.
4. SQLite. SQLite Documentation. Available at: https://www.sqlite.org/docs.html.
5. PostgreSQL Global Development Group. PostgreSQL Documentation. Available at: https://www.postgresql.org/docs/.
6. SQLAlchemy. SQLAlchemy 2.0 Documentation. Available at: https://docs.sqlalchemy.org/en/20/.
7. LM Studio. LM Studio as a Local LLM API Server. Available at: https://lmstudio.ai/docs/developer/core/server.
8. Docker. Running Containers. Available at: https://docs.docker.com/engine/containers/run/.
9. pytest Development Team. pytest Documentation. Available at: https://docs.pytest.org/en/stable/.
10. Microsoft. Playwright for Python Documentation. Available at: https://playwright.dev/python/docs/intro.

Security and Payment References

11. OWASP Foundation. Application Security Verification Standard. Available at: https://asvs.owasp.org/.
12. Razorpay. Web Standard Checkout Documentation. Available at: https://razorpay.com/docs/payments/payment-gateway/web-integration/standard/.

Internal Project Evidence

13. Panchal, H. V. ProjectForge AI Source Repository. Local project source, implementation modules, and generated applications.
14. ProjectForge Generated Commerce Security Review. Internal project report, 24 October 2026.
15. Security Repair Batch 1 - Verification and Remaining Work. Internal project report, 24 October 2026.

Access date for the web references: 26 October 2026.